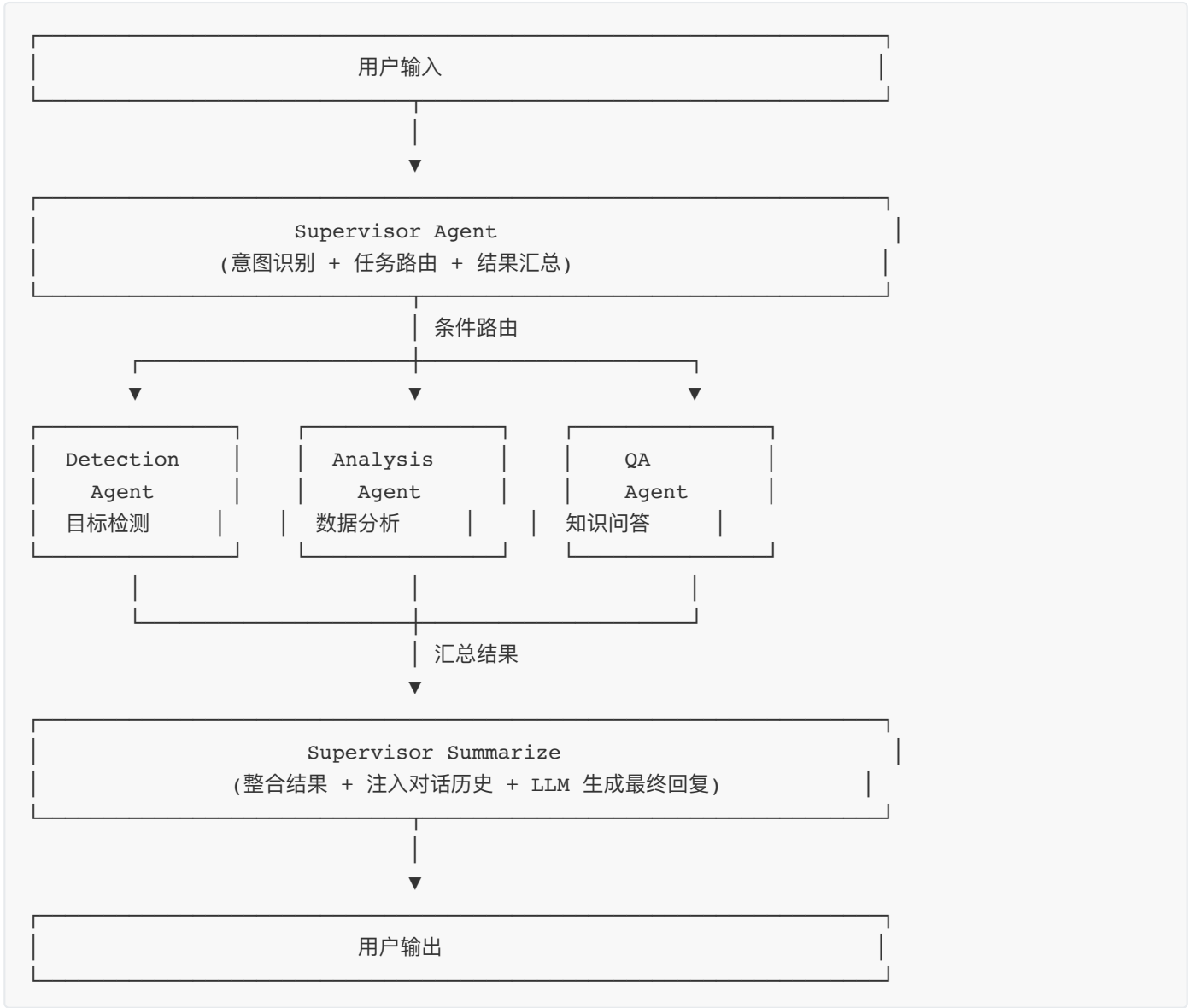


项目多智能体（Multi-Agent）实现思路

一、架构总览

项目采用 LangGraph 框架构建多智能体协作系统，遵循经典的 "Supervisor + Specialist Agents" 模式：



二、核心组件详解

2.1 共享状态定义 — state.py

职责：定义所有 Agent 共享的状态容器，在 LangGraph 状态图中流转。

```
class AgentState(TypedDict):  
    # 对话消息列表 (使用 add_messages reducer 自动追加)  
    messages: Annotated[list, add_messages]  
  
    # 路由决策  
    next_agent: str # "detection" | "analysis" | "qa"
```

```
# 各 Agent 的执行结果
detection_result: dict
analysis_result: dict
qa_result: str

# 最终回复
final_response: str

# 知识来源信息
knowledge_sources: list
has_knowledge: bool

# 用户信息
user_id: int
session_id: str
```

设计亮点：

字段	作用	说明
messages	对话历史	使用 add_messages reducer，自动追加新消息
next_agent	路由目标	Supervisor 写入，条件路由读取
*_result	各 Agent 结果	对应 Agent 写入，Summarize 读取
knowledge_sources	知识来源元数据	QA Agent 写入，前端展示用
user_id/session_id	用户/会话标识	贯穿整个流程，用于权限和记忆

2.2 Supervisor Agent — supervisor.py

职责：系统的"大脑"，负责意图识别、任务路由和结果汇总。

三大核心方法：

2.2.1 route() — 意图识别

```
def route(self, state: dict) -> dict:
    """路由：识别用户意图，决定交给哪个 Agent"""
    messages = [
        SystemMessage(content=SUPERVISOR_ROUTING_PROMPT),
        state["messages"][-1], # 最新用户消息
    ]
    response = self.llm.invoke(messages)
    next_agent = response.content.strip().lower()
    return {"next_agent": next_agent}
```

路由 Prompt （prompts.py#L82-L94）：

可选 Agent：

- detection：用户需要检测遥感图像中的目标，或询问检测结果，或要求重新检测
- analysis：用户需要查询检测历史、统计分析数据、查看使用报告
- qa：用户提出遥感领域知识问题（什么是 IoU、YOLO 原理等）

规则：

- 只回复一个 Agent 名称
- 如果用户要求重新检测、再检测一次，路由到 detection
- 如果无法判断，默认回复 qa

2.2.2 decide_next() — 条件路由函数

```
def decide_next(self, state: dict) -> str:
    """条件路由：根据 Supervisor 决策跳转"""
    next_agent = state.get("next_agent", "qa")
    if "detection" in next_agent:
        return "detection"
    elif "analysis" in next_agent:
        return "analysis"
    else:
        return "qa"
```

2.2.3 summarize() — 结果汇总

这是 Supervisor 最复杂的方法，负责：

1. 收集各 Agent 结果：检测结果、分析结果、知识问答结果
2. 处理知识来源：提取知识标题、去重、计算相似度
3. 注入对话历史：将历史消息格式化为"用户: xxx\nAI: xxx"
4. 动态选择 Prompt：根据是否有知识、是否有上下文选择不同模板
5. 生成最终回复：调用 LLM 生成回答

三种 Prompt 模板策略：

场景	Prompt 内容	适用条件
有知识库	知识内容 + 对话历史 + 当前问题	<code>has_knowledge=True</code>
有上下文	上下文信息 + 对话历史 + 当前问题	<code>context_parts</code> 非空
无知识无上下文	对话历史 + 当前问题	默认

2.3 专业 Agent 节点 — nodes.py

职责：三个专业化 Agent，每个处理特定类型的任务。

2.3.1 Detection Agent — 目标检测

```
def detection_node(state: Dict[str, Any]) -> Dict[str, Any]:
    last_message = messages[-1].content
    user_id = state.get("user_id", 1)

    # 根据消息中的附件标记选择工具
    if "[附件视频路径:" in last_message:
        result_str = detect_video_file.invoke({"video_path": ..., "user_id": user_id})
    elif "[附件多张图片路径:" in last_message:
        result_str = detect_batch_images.invoke({"image_paths": ..., "user_id": user_id})
    elif "[附件图片路径:" in last_message:
        if image_path.endswith(".zip"):
            result_str = detect_zip_images_file.invoke({"zip_path": ..., "user_id":
user_id})
        else:
            result_str = detect_single_image.invoke({"image_path": ..., "user_id":
user_id})

    return {"detection_result": result}
```

支持的检测类型:

检测类型	消息标记	工具函数
单图检测	[附件图片路径: xxx.jpg]	detect_single_image
批量检测	[附件多张图片路径: a.jpg,b.jpg]	detect_batch_images
ZIP检测	[附件图片路径: xxx.zip]	detect_zip_images_file
视频检测	[附件视频路径: xxx.mp4]	detect_video_file

2.3.2 Analysis Agent — 数据分析

```
def analysis_node(state: Dict[str, Any]) -> Dict[str, Any]:
    last_message = messages[-1].content

    # 根据关键词选择工具
    if "今天" in last_message or "最近" in last_message or "统计" in last_message:
        result_str = query_detection_stats.invoke({"days": 1})
    elif "历史" in last_message or "记录" in last_message:
        result_str = query_detection_history.invoke({"limit": 10})

    return {"analysis_result": result}
```

2.3.3 QA Agent — 知识问答

```
def qa_node(state: Dict[str, Any]) -> Dict[str, Any]:
    last_message = messages[-1].content

    # 调用 RAG 知识检索工具
    results_str = search_knowledge.invoke({"query": last_message})
    results = json.loads(results_str)

    return {"qa_result": results}
```

2.4 状态图构建 — graph.py

职责：使用 LangGraph 构建多 Agent 协作的状态机。

```
def build_agent_graph(llm, detection_agent_node, analysis_agent_node, qa_agent_node):
    supervisor = SupervisorAgent(llm)

    # 创建状态图
    workflow = StateGraph(AgentState)

    # 添加节点
    workflow.add_node("supervisor", supervisor.route)           # 路由节点
    workflow.add_node("detection", detection_agent_node)        # 检测节点
    workflow.add_node("analysis", analysis_agent_node)          # 分析节点
    workflow.add_node("qa", qa_agent_node)                      # 问答节点
    workflow.add_node("summarize", supervisor.summarize)        # 汇总节点

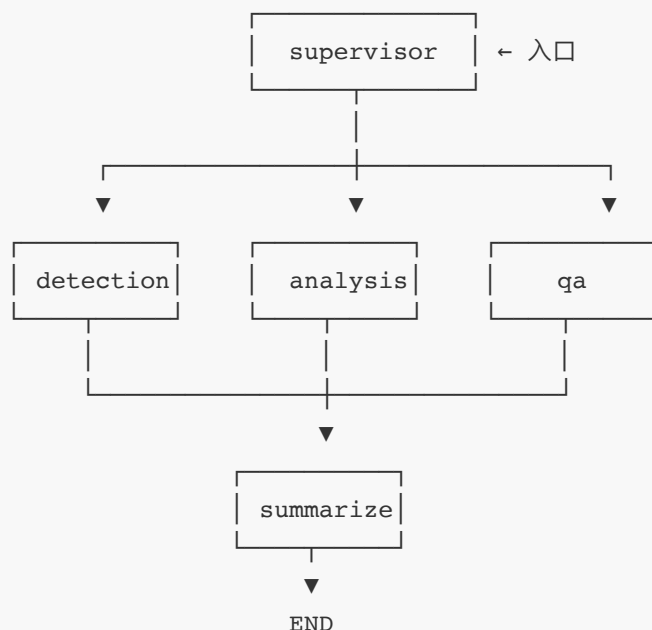
    # 设置入口
    workflow.set_entry_point("supervisor")

    # Supervisor 条件路由
    workflow.add_conditional_edges(
        "supervisor",
        supervisor.decide_next, # 路由函数
        {
            "detection": "detection",
            "analysis": "analysis",
            "qa": "qa",
        },
    )

    # 各 Agent 执行后进入汇总
    workflow.add_edge("detection", "summarize")
    workflow.add_edge("analysis", "summarize")
    workflow.add_edge("qa", "summarize")
    workflow.add_edge("summarize", END)

    return workflow.compile()
```

图结构：



三、完整调用流程

3.1 API 入口 — chat.py /multi-agent

```
async def multi_agent_chat(request: Request, current_user=Depends(get_current_user)):
    # 1. 解析请求参数
    body = await request.json()
    message = body.get("message", "")
    session_id = body.get("session_id") or str(uuid.uuid4())[0:8]

    # 2. 处理重新检测请求（从数据库查询历史检测任务）
    if any(keyword in message for keyword in ["再检测", "重新检测", "再试一次"]):
        # 查询最近检测任务 → 从 MinIO 下载原图 → 追加到消息中
        ...

    # 3. 加载对话历史（从 Redis）
    chat_history = []
    history = conversation_memory.load_history(current_user.id, session_id)
    for msg in history:
        if msg["role"] == "user":
            chat_history.append(HumanMessage(content=msg["content"]))
        elif msg["role"] == "ai":
            chat_history.append(AIMessage(content=msg["content"]))

    # 4. 保存用户消息
    conversation_memory.save_message(current_user.id, session_id, "user", message)

    # 5. 构建初始状态
    initial_state: AgentState = {
        "messages": chat_history + [HumanMessage(content=message)],
        "next_agent": "",
        "detection_result": {},
        "analysis_result": {},
```

```

    "qa_result": "",
    "final_response": "",
    "knowledge_sources": [],
    "has_knowledge": False,
    "user_id": current_user.id,
    "session_id": session_id,
}

```

6. 执行状态图 (流式)

```

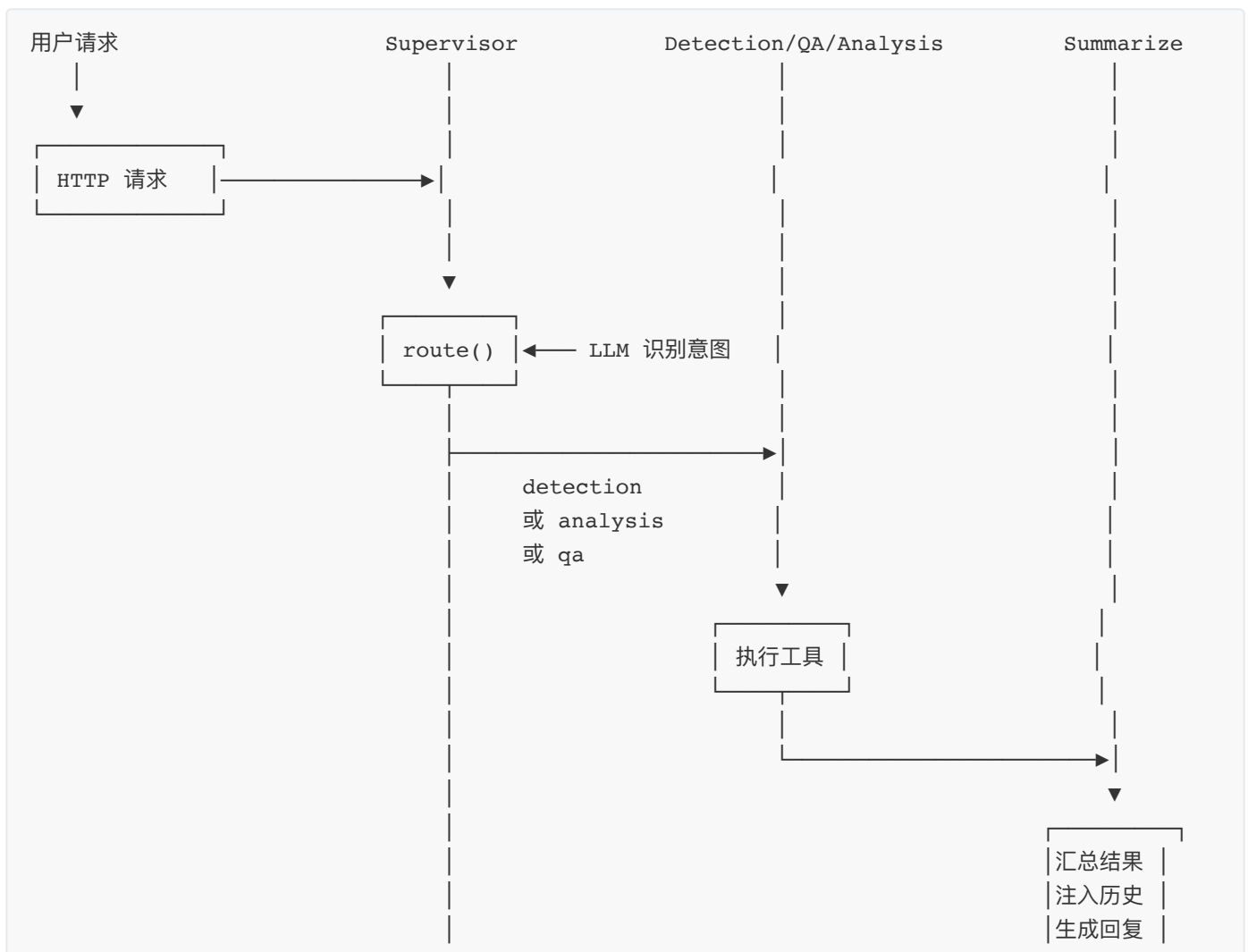
async for event in graph.astream(initial_state):
    for node, data in event.items():
        # 发送 multi_agent 事件 (前端可视化调用链)
        yield f"data: {{{'type': 'multi_agent', 'node': '{node}', ...}}}\n\n"

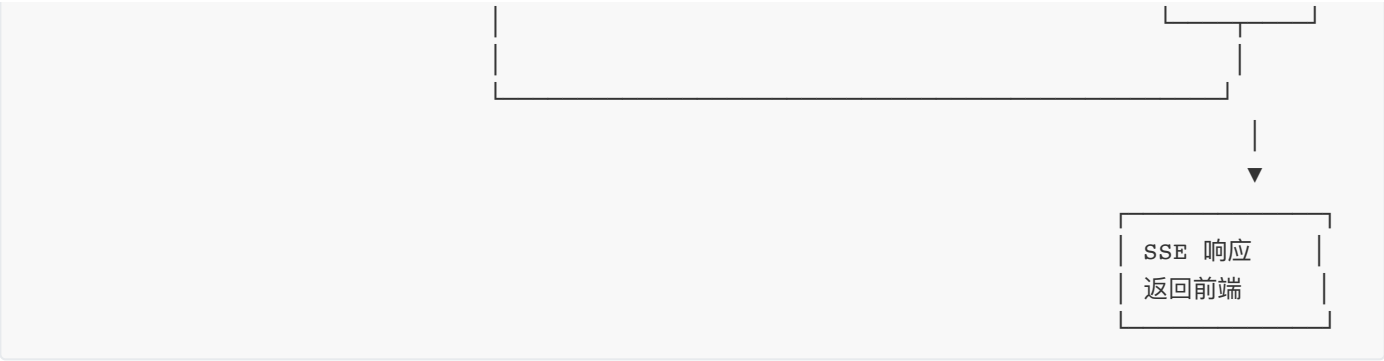
        if node == "summarize":
            # 发送 text_chunk 事件 (最终回复)
            yield f"data: {{{'type': 'text_chunk', 'content': ...}}}\n\n"

            # 保存 AI 回复到记忆
            conversation_memory.save_message(current_user.id, session_id, "ai",
final_response)

```

3.2 执行时序图





四、与单 Agent 架构的对比

维度	单 Agent (detection_agent.py)	多 Agent (LangGraph)
架构	单个 Agent 拥有所有工具，自主决策	Supervisor + 3 个 Specialist
路由	LLM 自主决定调用哪个工具	Supervisor 统一路由
工具数量	7 个（检测 4 + 分析 3，无知识库）	各节点按需调用，QA 节点专用知识库
记忆	集成 conversation_memory	通过 chat.py 注入历史
知识检索	不查询知识库（由多 Agent 的 QA 节点负责）	QA 节点强制查询知识库
调用链可视化	无	支持（multi_agent 事件）
适用场景	纯检测任务，响应快	复杂任务、知识问答、多轮对话

五、前端集成

5.1 调用链可视化 — ChatPage.vue

前端监听 `multi_agent` 事件，渲染多智能体调用链：

```

} else if (data.type === "multi_agent") {
  if (!lastMsg.agentChain) lastMsg.agentChain = [];
  lastMsg.agentChain.push({
    node: data.node,
    data: data.data,
  });
  // 解析检测结果
  if (data.data.detection_result) {
    lastMsg.detectionResult = data.data.detection_result;
  }
}
}
```

可视化效果：

多智能体调用链

[Supervisor] → [Detection Agent] → [Summarize]

Supervisor: 路由决策: detection
Detection Agent: 检测到 5 个目标
Summarize: 生成最终回复

六、关键设计决策总结

设计点	选择	理由
架构	LangGraph	原生支持状态机、条件路由、流式执行
路由方式	LLM 驱动 (Supervisor)	灵活处理自然语言意图，可扩展
状态传递	TypedDict + add_messages	类型安全，自动消息追加
工具调用	<code>.invoke()</code> 方法	避免 <code>@tool</code> 装饰器的参数错误
记忆注入	Supervisor 汇总阶段	统一管理，避免重复注入
知识检索	QA 节点专用	检测任务不查询知识库，提高效率
调用链追踪	SSE 事件实时推送	前端可视化，提升用户体验
重新检测	API 层处理	从数据库查询历史任务，不依赖 Agent

七、架构优势

1. 职责分离：每个 Agent 专注于单一领域，易于维护和扩展

2. 灵活路由：基于 LLM 的意图识别，支持自然语言交互

3. 可观测性：完整的调用链追踪，便于调试和优化

4. 可扩展性：新增 Agent 只需添加节点和路由规则

5. 记忆增强：对话历史在汇总阶段统一注入，保证上下文一致性

6. 流式响应：SSE 实时推送中间状态，提升交互体验

八、潜在优化方向

优化项	实现思路
多 Agent 并行执行	支持同时调用多个 Agent（如检测 + 分析）
动态 Agent 注册	支持运行时添加/移除 Agent
重试机制	Agent 执行失败时自动重试或降级
缓存机制	相同问题的检测结果或知识检索结果缓存
工具选择优化	为每个 Agent 定制工具列表，减少决策负担
路由优化	结合规则引擎 + LLM，提高路由准确率

九、多 Agent 并行执行示例：检测 + 知识问答并行

一、场景说明

用户请求：“帮我检测这张图片中的飞机目标，同时解释一下什么是 IoU？”

并行任务：

- 1. Detection Agent：检测图像中的飞机目标
- 2. QA Agent：检索知识库中关于 IoU 的知识

预期效果：两个 Agent 同时执行，总响应时间 ≈ max(检测耗时, 知识检索耗时)

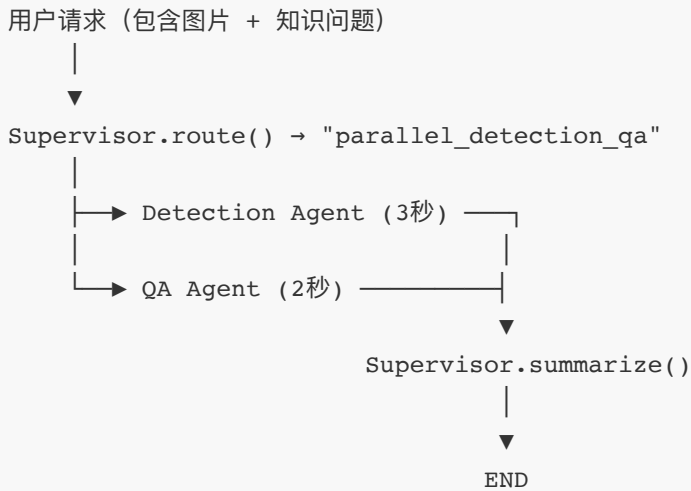
二、串行 vs 并行对比

2.1 当前串行架构



总耗时：检测 3秒 + 问答 2秒 = 5秒（需要两条消息）

2.2 并行架构



总耗时：max(检测 3秒, 问答 2秒) = **3秒** （一条消息完成）

三、并行实现流程详解

3.1 第一步：扩展 AgentState

在 `state.py` 中添加并行任务标记：

```
class AgentState(TypedDict):
    # 原有字段...

    # 并行任务标记
    parallel_tasks: list[str] # 例如 ["detection", "qa"]
    parallel_results: dict    # 并行任务结果
```

3.2 第二步：扩展路由 Prompt

在 `prompts.py` 中添加并行路由规则：

```
SUPERVISOR_PARALLEL_ROUTING_PROMPT = """你是一个任务调度器。根据用户输入，选择最合适的 Agent 处理。

可选任务：
- detection：用户需要检测遥感图像中的目标
- analysis：用户需要查询检测历史、统计分析数据
- qa：用户提出遥感领域知识问题

规则：
1. 如果用户请求同时包含图像和知识问题，返回多个任务，用逗号分隔
   例如："detection,qa"（同时检测和问答）
2. 如果只包含图像，返回 "detection"
3. 如果只包含知识问题，返回 "qa"
4. 如果只包含统计请求，返回 "analysis"
5. 如果无法判断，默认返回 "qa"
6. 只回复任务名称，不要回复其他内容
"""
```

3.3 第三步：创建并行节点

在 nodes.py 中添加并行执行节点：

```
import asyncio

def parallel_detection_qa_node(state: Dict[str, Any]) -> Dict[str, Any]:
    """并行执行检测和知识问答"""
    messages = state["messages"]
    last_message = messages[-1].content
    user_id = state.get("user_id", 1)

    # 异步并行执行两个任务
    async def run_parallel():
        # 任务1: 检测
        detection_task = asyncio.create_task(
            asyncio.to_thread(_run_detection, last_message, user_id)
        )

        # 任务2: 知识问答
        qa_task = asyncio.create_task(
            asyncio.to_thread(_run_qa, last_message)
        )

        # 等待两个任务完成
        detection_result, qa_result = await asyncio.gather(
            detection_task,
            qa_task
        )

        return {
            "detection_result": detection_result,
            "qa_result": qa_result,
        }

    # 运行并行任务
    loop = asyncio.get_event_loop()
    results = loop.run_until_complete(run_parallel())

    return results

def _run_detection(message: str, user_id: int) -> dict:
    """执行检测任务（同步函数）"""
    if "[附件图片路径:" in message:
        image_path = message.split("[附件图片路径:")[1].split(")")[0].strip()
        result_str = detect_single_image.invoke(
            {"image_path": image_path, "user_id": user_id}
        )
        return json.loads(result_str)
    return {}
```

```
def _run_qa(message: str) -> dict:
    """执行知识问答（同步函数）"""
    results_str = search_knowledge.invoke({"query": message})
    return json.loads(results_str)
```

3.4 第四步：构建并行状态图

修改 `graph.py`，添加并行分支：

```
from langgraph.graph import END, StateGraph
from langgraph.prebuilt import ToolExecutor

def build_parallel_agent_graph(llm, detection_node, analysis_node, qa_node,
                              parallel_node):
    """构建支持并行执行的多 Agent 状态图"""
    from app.agent.supervisor import SupervisorAgent

    supervisor = SupervisorAgent(llm)

    workflow = StateGraph(AgentState)

    # 添加节点
    workflow.add_node("supervisor", supervisor.route)
    workflow.add_node("detection", detection_node)
    workflow.add_node("analysis", analysis_node)
    workflow.add_node("qa", qa_node)
    workflow.add_node("parallel_detection_qa", parallel_node) # 新增并行节点
    workflow.add_node("summarize", supervisor.summarize)

    # 设置入口
    workflow.set_entry_point("supervisor")

    # Supervisor 条件路由（支持多任务）
    workflow.add_conditional_edges(
        "supervisor",
        supervisor.decide_next_parallel, # 新的路由函数
        {
            "detection": "detection",
            "analysis": "analysis",
            "qa": "qa",
            "detection,qa": "parallel_detection_qa", # 并行任务
            "detection,analysis": "parallel_detection_analysis",
        },
    )

    # 各节点执行后进入汇总
    workflow.add_edge("detection", "summarize")
    workflow.add_edge("analysis", "summarize")
    workflow.add_edge("qa", "summarize")
    workflow.add_edge("parallel_detection_qa", "summarize") # 并行节点也进入汇总
```

```
workflow.add_edge("summarize", END)

return workflow.compile()
```

3.5 第五步：扩展 Supervisor 路由函数

在 `supervisor.py` 中添加并行路由逻辑：

```
def decide_next_parallel(self, state: dict) -> str:
    """条件路由：支持并行任务"""
    next_agent = state.get("next_agent", "qa")

    # 检查是否包含多个任务
    if "," in next_agent:
        tasks = [t.strip() for t in next_agent.split(",")]
        # 排序保证一致性
        tasks.sort()
        return ",".join(tasks)

    # 单任务路由
    if "detection" in next_agent:
        return "detection"
    elif "analysis" in next_agent:
        return "analysis"
    else:
        return "qa"
```

四、完整执行时序

用户请求："检测这张图片中的飞机，解释一下什么是 IoU？"

↓

POST /api/chat/multi-agent

- 解析请求：包含图片 + 知识问题
- 加载对话历史 (Redis)
- 保存用户消息 (Redis)

↓

构建初始状态：AgentState

↓

graph.astream(initial_state)

↓

节点：supervisor
操作：Supervisor.route()
输入："检测这张图片中的飞机，解释一下什么是 IoU？"
LLM 输出："detection,qa"

状态更新: next_agent = "detection,qa"



条件路由: decide_next_parallel()

判断: "detection,qa" → "parallel_detection_qa"



节点: parallel_detection_qa

操作: asyncio.gather()



等待所有任务完成 (3秒)

状态更新:

detection_result = {...}

qa_result = {"knowledge": [...], ...}



节点: summarize

操作: Supervisor.summarize()

收集结果:

- 检测结果: 发现 5 架飞机
- 知识内容: IoU = 交并比, 衡量检测框精度...

注入对话历史

LLM 生成最终回复:

"图片中检测到 5 架飞机。\\n\\n"

"IoU (交并比) 是衡量目标检测精度的指标..."

状态更新:

final_response = "..."

knowledge_sources = [...]

has_knowledge = True



SSE 流式输出:

{"type": "multi_agent", "node": "supervisor", ...}

```
{ "type": "multi_agent", "node": "parallel_detection_qa", ... }
{ "type": "multi_agent", "node": "summarize", ... }
{ "type": "text_chunk", "content": "...", ... }
{ "type": "[DONE]" }
|
▼
```

保存 AI 回复到 Redis 对话记忆

五、关键技术点

5.1 异步并行执行

```
async def run_parallel():
    # 创建两个异步任务
    task1 = asyncio.create_task(asyncio.to_thread(run_detection))
    task2 = asyncio.create_task(asyncio.to_thread(run_qa))

    # 等待所有任务完成
    result1, result2 = await asyncio.gather(task1, task2)

    return {"detection_result": result1, "qa_result": result2}
```

5.2 LangGraph 条件路由

```
workflow.add_conditional_edges(
    "supervisor",
    supervisor.decide_next_parallel,
    {
        "detection": "detection",
        "qa": "qa",
        "detection,qa": "parallel_detection_qa", # 映射到并行节点
    },
)
```

5.3 状态共享与汇总

并行节点的结果会写入 `AgentState` 的对应字段，然后在 `summarize` 节点中统一收集：


```
def summarize(self, state: dict) -> dict:
    context_parts = []

    # 检测结果
    if state.get("detection_result"):
        context_parts.append(f"检测结果: {state['detection_result']}")

    # 知识问答结果
    if state.get("qa_result"):
        context_parts.append(f"问答结果: {state['qa_result']}")

    # 构建 Prompt 并生成回复
    ...
```

六、适用场景

场景	并行任务	预期收益
检测 + 知识	Detection + QA	减少等待时间
检测 + 统计	Detection + Analysis	检测完成即显示统计
全链路分析	Detection + Analysis + QA	一站式分析报告

七、注意事项

- 1. 任务独立性：并行任务之间不能有依赖关系（如检测结果不能作为 QA 的输入）
- 2. 错误处理：单个任务失败不应影响

应用场景

用户请求：上传一张遥感图片，并询问 "检测这张图片，同时解释一下什么是飞机目标？"

串行执行（当前架构）：

```
Supervisor -> Detection (5秒) -> Summarize -> QA (2秒) -> Summarize -> 回复
总耗时：约 7 秒
```

并行执行（优化后）：

```
Supervisor -> [Detection (5秒) + QA (2秒)] 并行 -> Summarize -> 回复
总耗时：约 5 秒（取最长任务）
```